Journal of Machine Learning Research XX (2026) 1-7 Submitted MM/26; Published MM/YY

torchtsetlin: Fast, Interpretable Tsetlin Machines for the PyTorch Ecosystem

Vajira Thambawita VAJIRA@SIMULA.NO
Simula Metropolitan Center for Digital Engineering
Oslo, Norway

Editor: TBD

Abstract

The Tsetlin machine learns propositional rules with teams of Tsetlin automata that are driven by reward and penalty rather than gradients, so its models are interpretable by construction and cheap to evaluate. Its software, however, consists of C and CUDA kernels behind scikit-learn-style wrappers that sit outside the deep-learning ecosystem. torchtsetlin implements the Tsetlin machine family as ordinary PyTorch modules: vanilla and weighted, coalesced, regression, convolutional over sequences, images and volumes, and dense per-pixel segmentation. The automata are a tensor buffer, so a model moves between devices with `.to(device)`, is saved with `state_dict()`, consumes a `DataLoader`, and learns from a mini-batch with a single `update(x, y)` call. The library's core technique is a batched formulation of Type I and Type II feedback: the per-example automaton transitions become event counts computed with matrix products and realised as binomial draws, with one commit per mini-batch. On a single GPU this trains up to $X\times$ faster than the same code on a multi-core CPU and $Y\times$ faster than existing Tsetlin machine packages, and it matches the sequential algorithm at moderate batch sizes. The package is pure Python, MIT-licensed and on PyPI.

Keywords: Tsetlin machine, interpretable machine learning, propositional logic, PyTorch, GPU

1 Introduction

A Tsetlin machine (TM; Granmo, 2018) represents each class by conjunctive clauses over Boolean features and their negations. A clause such as $x_0 \wedge \neg x_1$ votes for or against its class, and the largest vote sum wins. Each (clause, literal) pair is a Tsetlin automaton (Tsetlin, 1961; Narendra and Thathachar, 1989), a small counter that decides whether the literal is included. Learning is stochastic reward and penalty (Type I and Type II feedback) instead of gradients, so the model is a readable rule set (Rudin, 2019) and inference is bitwise, suited to low-power hardware (Wheeldon et al., 2021; Maheshwari et al., 2023; Lin et al., 2025). The family now includes convolutional (Granmo et al., 2019), weighted (Phoulady et al., 2019), regression (Abeyrathna et al., 2020) and coalesced machines (Glimsdal and Granmo, 2021), with regularisers such as clause-size budgets (Abeyrathna et al., 2023) and drop clause (Sharma et al., 2023).

The reference software implements each variant as a C or CUDA kernel behind a `fit`/`predict` interface (Granmo, 2024, 2021; Granmo and Andersen, 2024). For PyTorch (Paszke et al., 2019) users this leaves four gaps: *(i)* the learned state sits in a private buffer that

©2026 Vajira Thambawita.

License: CC-BY 4.0, see `https://creativecommons.org/licenses/by/4.0/`. Attribution requirements are provided at `http://jmlr.org/papers/vXX/YY-NNNN.html`.

tensor tools cannot inspect, mask or save; *(ii)* the device is fixed by the imported package, so CPU and GPU runs follow different code paths; *(iii)* Booleanised data must leave the framework as NumPy arrays; and *(iv)* a new feedback rule or output layer means a new kernel. torchtsetlin closes these gaps with tensor-state `nn.Modules`, batched GPU feedback, one learning loop for flat, convolutional and dense inputs, and an end-to-end workflow from Booleanisation to rules.

2 Design

State as tensors. A machine with C clauses over F features holds an integer buffer `ta_state` of shape $(C, 2F)$; literal k is included in clause j when its counter reaches the memory depth N. The counters are buffers, not parameters, so `to()`, `state_dict()` and lazy shape inference (as in `LazyLinear`) work unchanged. Learning is `update(x, y)`, the analogue of `loss.backward(); optimizer.step()`, which returns the pre-update vote sums for logging; the empty-clause convention (output 1 while learning, 0 while predicting) follows `train()`/`eval()`.

One learning loop, many input structures. The package has four layers. `functional` holds all learning maths as stateless tensor operations, and `TsetlinMachineBase` runs the generic loop. A concrete model supplies only an output layer and a feedback policy through three hooks. A new input structure overrides encoding, clause evaluation and feedback counting: one mixin turns every flat model into a convolutional one, in which a clause fires if it matches any patch and positions are thermometer-encoded (Granmo et al., 2019); a 3-D subclass changes only the patch geometry; and dense segmentation folds pixels into the batch axis so that each is a flat example. No variant re-implements learning.

Batched feedback. Classically, every training example triggers a stochastic pass over all $C \times 2F$ automata. For a mini-batch of B examples, torchtsetlin evaluates all clauses at once and draws the per-example clause selections for feedback with the usual probability $(T \mp \mathrm{clip}(v, -T, T))/2T$ for vote sum v and margin T. With $S_{\mathrm{I}}, S_{\mathrm{II}} \in \{0,1\}^{B \times C}$ the Type I and Type II selections of firing clauses, $\bar{S}_{\mathrm{I}}$ the Type I selections of non-firing clauses and $L \in \{0,1\}^{B \times 2F}$ the literals, the feedback events are counted with matrix products,

$$n^{\mathrm{true}} = S_{\mathrm{I}}^{\top} L, \quad n^{\mathrm{false}} = S_{\mathrm{I}}^{\top}(1-L), \quad n^{\mathrm{II}} = S_{\mathrm{II}}^{\top}(1-L), \quad n^{\mathrm{Ib}} = \bar{S}_{\mathrm{I}}^{\top}\mathbf{1}.$$

Each event is an independent Bernoulli trial, so the aggregate transition is drawn in closed form and committed once per batch:

$$\Delta = \mathrm{Bin}\big(n^{\mathrm{true}}, p_{\mathrm{mem}}\big) - \mathrm{Bin}\big(n^{\mathrm{false}} + n^{\mathrm{Ib}}\mathbf{1}^{\top}, 1/s\big) + \min\big(n^{\mathrm{II}}, \max(N - \mathrm{state}, 0)\big),$$

with specificity s and $p_{\mathrm{mem}} = (s-1)/s$, or 1 with boosted true-positive feedback. At $B = 1$ this is the classical update term by term; for $B > 1$ it approximates it as mini-batch SGD approximates SGD (Bottou et al., 2018), and `feedback_mode="sequential"` keeps the exact path. Because one commit can include many literals, the clause-size budget (Abeyrathna et al., 2023) is a hard cap enforced at commit.

Listing 1: Training, evaluating and reading a Tsetlin machine on Noisy XOR.

```
import torchtsetlin as tt
x_train, y_train = tt.data.make_noisy_xor(5000, noise=0.4)   # Boolean features, int labels
x_test,  y_test  = tt.data.make_noisy_xor(2000, noise=0.0, seed=1)
model = tt.TsetlinMachine(n_features=12, n_classes=2, n_clauses=20, T=15, s=3.9).to("cuda")
trainer = tt.Trainer(model, batch_size=10, callbacks=[tt.EarlyStopping(patience=20)])
trainer.fit((x_train, y_train), epochs=100, val_data=(x_test, y_test))
print(trainer.evaluate((x_test, y_test)))   # {'accuracy': ...}
print(model.rules()[:2])  # ['IF x0 AND NOT x1 THEN 1', 'IF x1 AND NOT x0 THEN 1']
```

3 Functionality and Usage

Listing 1 shows the complete workflow on Noisy XOR (Granmo, 2018): with 40% label noise the machine recovers and prints the generating rule. Version [0.3.x] ships `TsetlinMachine` (optional clause weights (Phoulady et al., 2019)), `CoalescedTsetlinMachine` (multi-class or multi-label) and `RegressionTsetlinMachine`, convolutional 1-D, 2-D and 3-D variants of each, and dense segmentation machines. Options include drop clause and drop literal and focused negative sampling. Thermometer, one-hot, bit-plane, adaptive-threshold, colour, hypervector (Halenka et al., 2024) and pyramid encoders are `nn.Module`s that run on the model's device and carry feature names into the rules (`worst_perimeter>=105.9` rather than `x37`). A `Trainer` with callbacks accepts tensors and `DataLoader`s, and the interpretation tools extract rules and global, local and per-pixel importance (Blakely and Granmo, 2021). The package is pure Python on PyTorch $\geq$ 2.0 and NumPy, with no compiler or CUDA toolkit, for Python 3.9–3.12.

4 Comparison with Existing Software

Table 1 compares torchtsetlin with the maintained TM packages. `pyTsetlinMachine` (Granmo, 2024) and `PyTsetlinMachineCUDA` (Granmo, 2021) implement one class per variant in C and CUDA; `tmu` (Granmo and Andersen, 2024) unifies them over reusable clause and weight banks with CPU and CUDA backends; `green-tsetlin` (Glimsdal, 2025; Glimsdal et al., 2024) targets efficient CPU training and deployment. Every GPU path among them is a hand-written PyCUDA kernel. All keep the automata in a native buffer; torchtsetlin alone builds on a general-purpose tensor library, trading some kernel-level speed on small CPU workloads for device portability, interoperability and extensibility in Python.

5 Performance and Fidelity

Batching amortises the cost of an `update`, which touches every automaton. For a 784-feature machine with 500 clauses per class (Figure 1a), GPU throughput grows from a examples/s at $B = 1$ to b at $B = 4096$; at $B = 256$ the same code runs $c\times$ faster on an [RTX 3090] than on a [16-core CPU], up to $e\times$ for the clause counts of the image literature. Batching is an approximation, so Figure 1b measures its cost: on Noisy XOR, batches up to about 50 stay in the accuracy band of the exact sequential algorithm and batch 200 degrades, which motivates the recommended 16–64 (flat) and 8–32 (convolutional). Accuracy matches published TM results (MNIST x%, Fashion-MNIST y%, CIFAR-10 z%; LeCun et al., 1998;

Table 1: Comparison with TM software, latest PyPI releases [as of submission] (✓ yes, ○ partial, ✗ no; red: not yet verified or measured). Bottom: MNIST, 2000 clauses, batch 256; hardware.

	`pyTsetlinMachine`	`PyTMCUDA`	`tmu`	`green-tsetlin`	`torchtsetlin`
Backend	C	PyCUDA	C / PyCUDA	C++	PyTorch
CPU and GPU from one class	✗	✗	✓	✗	✓
Conv. 1-D / 2-D / 3-D	✗✓✗	✗✓✗	✗✓✗	[?]	✓✓✓
Regression / coalesced	✓✗	✓✗	✓✓	✗✗	✓✓
Dense segmentation	✗	✗	✗	✗	✓
Tensor state, `state_dict`	✗	✗	✗	✗	✓
New variant without C/CUDA	✗	✗	○	✗	✓
Unit tests and CI	✗	✗	✓	✓	✓
MNIST epoch, CPU / GPU (s)	0.0 / –	– / 0.0	0.0 / 0.0	0.0 / –	0.0 / 0.0
Peak GPU memory (MiB)	–	0	0	–	0
MNIST test accuracy (%)	0.0	0.0	0.0	0.0	0.0

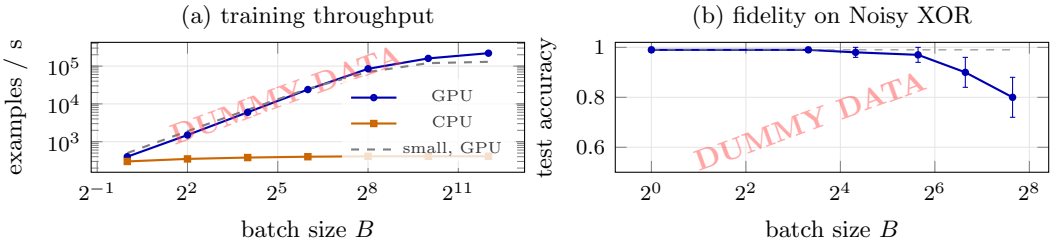


Figure 1: (a) Training throughput, same code on GPU and CPU (784 features, 500 clauses per class; dashed: 12 features, 10 clauses). (b) Noisy XOR accuracy, mean ± s.d. over 3 seeds; $B = 1$ is the exact sequential algorithm.

Xiao et al., 2017; Krizhevsky, 2009). One command, `benchmarks/run_benchmarks.sh`, reproduces every number and figure on other hardware.

6 Software Quality and Community

The suite has n tests with $p\%$ line coverage; device-dependent tests run on the CPU and on CUDA when present. Beyond the maths, it pins behaviour that breaks silently, such as empty-clause semantics, the hard clause budget under batched commits, RNG-free convolutional prediction and the bit-identical reduction of a depth-1 3-D model to the 2-D one. Continuous integration lints and tests on Python 3.9, 3.11 and 3.12 [on Linux, macOS and Windows]; tagged releases go to PyPI by Trusted Publishing. The documentation[1] offers guides, examples, an API reference, and an SQL-queryable catalogue of r results on d datasets from m TM papers. Development is open on GitHub (`https://github.com/vlbthambawita/torchtsetlin`, MIT licence).

1. `https://vajira.info/torchtsetlin/`

Acknowledgments and Disclosure of Funding

[Funding sources and grant numbers.] The author declares no competing interests.

References

K. Darshana Abeyrathna, Ole-Christoffer Granmo, Xuan Zhang, Lei Jiao, and Morten Goodwin. The regression Tsetlin machine: A novel approach to interpretable nonlinear regression. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 378(2164):20190165, 2020. doi: 10.1098/rsta.2019.0165.

K. Darshana Abeyrathna, Ahmed A. O. Abouzeid, Bimal Bhattarai, Charul Giri, Sondre Glimsdal, Ole-Christoffer Granmo, Lei Jiao, Rupsa Saha, Jivitesh Sharma, Svein A. Tunheim, and Xuan Zhang. Building concise logical patterns by constraining Tsetlin machine clause size. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3395–3403, 2023. doi: 10.24963/ijcai.2023/378.

Christian D. Blakely and Ole-Christoffer Granmo. Closed-form expressions for global and local interpretation of Tsetlin machines. In *Advances and Trends in Artificial Intelligence. Artificial Intelligence Practices (IEA/AIE 2021)*, volume 12798 of *Lecture Notes in Computer Science*, pages 158–172. Springer, 2021. doi: 10.1007/978-3-030-79457-6_14.

Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018. doi: 10.1137/16M1080173.

Sondre Glimsdal. green-tsetlin: A fast Tsetlin machine implementation, intended for production. `https://github.com/ooki/green_tsetlin`, 2025. Version 1.1.0 on PyPI (August 2025); C++ core with Python bindings; MIT licence.

Sondre Glimsdal and Ole-Christoffer Granmo. Coalesced multi-output Tsetlin machines with clause sharing. arXiv preprint arXiv:2108.07594, 2021. URL `https://arxiv.org/abs/2108.07594`.

Sondre Glimsdal, Sebastian Østby, Tobias M. Brambo, and Eirik M. Vinje. Green Tsetlin: Redefining efficiency in Tsetlin machine frameworks. arXiv preprint arXiv:2405.04212, 2024. URL `https://arxiv.org/abs/2405.04212`.

Ole-Christoffer Granmo. The Tsetlin machine – a game theoretic bandit driven approach to optimal pattern recognition with propositional logic. arXiv preprint arXiv:1804.01508, 2018. URL `https://arxiv.org/abs/1804.01508`.

Ole-Christoffer Granmo. PyTsetlinMachineCUDA: Massively parallel and asynchronous architecture for logic-based AI. `https://github.com/cair/PyTsetlinMachineCUDA`, 2021. Version 0.1.9 on PyPI (June 2021); PyCUDA kernels; MIT licence.

Ole-Christoffer Granmo. pyTsetlinMachine: Tsetlin machine, convolutional, regression, weighted and embedding Tsetlin machines in C with a Python interface. `https://github.com/cair/pyTsetlinMachine`, 2024. Version 0.6.6 on PyPI (March 2024); MIT licence.

Ole-Christoffer Granmo and Per-Arne Andersen. TMU: Tsetlin machine unified. https://github.com/cair/tmu, 2024. Version 0.8.3 on PyPI (March 2024); MIT licence.

Ole-Christoffer Granmo, Sondre Glimsdal, Lei Jiao, Morten Goodwin, Christian W. Omlin, and Geir Thore Berge. The convolutional Tsetlin machine. arXiv preprint arXiv:1905.09688, 2019. URL https://arxiv.org/abs/1905.09688.

Vojtech Halenka, Ahmed K. Kadhim, Paul F. A. Clarke, Bimal Bhattarai, Rupsa Saha, Ole-Christoffer Granmo, Lei Jiao, and Per-Arne Andersen. Exploring effects of hyperdimensional vectors for Tsetlin machines. In *2024 International Symposium on the Tsetlin Machine (ISTM)*, pages 1–8. IEEE, 2024. doi: 10.1109/ISTM62799.2024.10931546.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. URL https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

Baizhou Lin, Yuetong Fang, Renjing Xu, Rishad Shafik, and Jagmohan Chauhan. TsetlinKWS: A 65nm 16.58μW, 0.63mm^2 state-driven convolutional Tsetlin machine-based accelerator for keyword spotting. arXiv preprint arXiv:2510.24282, 2025. URL https://arxiv.org/abs/2510.24282.

Sidharth Maheshwari, Tousif Rahman, Rishad Shafik, Alex Yakovlev, Ashur Rafiev, Lei Jiao, and Ole-Christoffer Granmo. REDRESS: Generating compressed models for edge inference using Tsetlin machines. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(9):11152–11168, 2023. doi: 10.1109/TPAMI.2023.3268415.

Kumpati S. Narendra and Mandayam A. L. Thathachar. *Learning Automata: An Introduction*. Prentice Hall, Englewood Cliffs, NJ, 1989. ISBN 0-13-485558-2.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pages 8024–8035, 2019.

Adrian Phoulady, Ole-Christoffer Granmo, Saeed Rahimi Gorji, and Hady Ahmady Phoulady. The weighted Tsetlin machine: Compressed representations with weighted clauses. arXiv preprint arXiv:1911.12607, 2019. URL https://arxiv.org/abs/1911.12607. Presented at the Ninth International Workshop on Statistical Relational AI (StarAI 2020).

Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019. doi: 10.1038/s42256-019-0048-x.

Jivitesh Sharma, Rohan Yadav, Ole-Christoffer Granmo, and Lei Jiao. Drop clause: Enhancing performance, robustness and pattern recognition capabilities of the Tsetlin machine. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 13547–13555, 2023. doi: 10.1609/aaai.v37i11.26588.

Michael L. Tsetlin. On behaviour of finite automata in random medium. *Avtomatika i Telemekhanika*, 22(10):1345–1354, 1961. In Russian. English translation in *Automation and Remote Control*, 22:1210–1219, 1961.

Adrian Wheeldon, Alex Yakovlev, Rishad Shafik, and Jordan Morris. Low-latency asynchronous logic design for inference at the edge. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 370–373. IEEE, 2021. doi: 10.23919/DATE51398.2021.9474126. arXiv:2012.03402 (2020).

Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747, 2017. URL `https://arxiv.org/abs/1708.07747`.